



Mata Kuliah : Pemrograman Web Lanjut
Program Studi : D4 – Teknik Informatika
Semester : 4

Kelas : TI-2D
NIM : 244107020142
Nama : Faatihurrizki Prasojo
Jobsheet Ke- : 3

Laporan Jobsheet

Praktikum Ke-1

Langkah	Jawaban/Deskripsi
1	Buat file migrasi untuk table m_level dengan perintah <pre>Php artisan make:migration create_m_level_table --create=m_level</pre> <pre>/** * Run the migrations. */ public function up(): void { Schema::create('m_level', function (Blueprint \$table) { \$table->id(); \$table->timestamps(); }); }</pre>
2	Modifikasi sesuai dengan desain database yang sudah ada



	<pre>return new class extends Migration { /** * Run the migrations. */ public function up(): void { Schema::create('m_level', function (Blueprint \$table) { \$table->id('level_id'); \$table->string('level_kode', 10)->unique(); \$table->string('level_nama', 100); \$table->timestamps(); }); } /** * Reverse the migrations. */ public function down(): void { Schema::dropIfExists('m_level'); } };</pre>
3	kemudian jalankan perintah ini pada terminal VSCode untuk melakukan migrasi <pre>PS C:\laragon\www\JS3> php artisan migrate INFO: Preparing database. Creating migration table 418.58ms DONE Creating migration table 418.58ms DONE Creating migration table 418.58ms DONE INFO: Running migrations. 0001_01_01_000000_create_users_table 1s DONE 0001_01_01_000001_create_cache_table 180.44ms DONE 0001_01_01_000002_create_jobs_table 633.06ms DONE 2026_02_24_061859_create_m_level_table 156.82ms DONE</pre>
4	Cek database di phpMyAdmin 
5	Buat table database dengan migration untuk table m_kategori yang sama-sama tidak memiliki foreign key <pre>PS C:\laragon\www\JS3> php artisan make:migration create_m_kategori_table --create=m_kategori INFO: Migration [C:\laragon\www\JS3\database\Migrations\2026_03_03_090229_create_m_kategori_table.php] created successfully.</pre>



```
return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('m_kategori', function (Blueprint $table) {
            $table->id('kategori_id');
            $table->string('kategori_kode', 10)->unique();
            $table->string('kategori_nama', 100);
            $table->timestamps(); // Menghasilkan kolom created_at dan updated_at
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('m_kategori');
    }
};
```

```
PS C:\laragon\www\JS3> php artisan migrate
```

```
INFO Running migrations.
```

```
2026_03_03_090229_create_m_kategori_table . 93.32ms DONE
```

Praktikum Ke-2

Langkah	Jawaban/Deskripsi
1	Buka terminal VSCode kalian, dan buat file migrasi untuk table m_user <pre>PS C:\laragon\www\JS3> php artisan make:migration create_m_user_table --table=m_user</pre> <pre>INFO Migration [c:\laragon\www\JS3\database\Migrations\2026_03_03_084419_create_m_user_table.php] created successfully.</pre>
2	Buka file migrasi untuk table m_user, dan modifikasi seperti berikut



	<pre>public function up(): void { Schema::create('m_user', function (Blueprint \$table) { \$table->id('user_id'); \$table->unsignedBigInteger('level_id')->index(); \$table->string('username', 20)->unique(); \$table->string('nama', 100); \$table->string('password'); \$table->timestamps(); // Relasi Foreign Key \$table->foreign('level_id')->references('level_id')->on('m_level'); }); }</pre>
3	Jalankan perintah php artisan migrate <pre>PS C:\laragon\www\JS3> php artisan migrate INFO Running migrations. 2026_03_03_084419_create_m_user_table 7.03ms DONE</pre>
4	Buat table database dengan migration untuk table-table yang memiliki foreign key a. m_barang <pre>return new class extends Migration { /** * Run the migrations. */ public function up(): void { Schema::create('m_barang', function (Blueprint \$table) { \$table->id('barang_id'); // Primary Key \$table->unsignedBigInteger('kategori_id')->index(); // Foreign Key ke tabel kat \$table->string('barang_kode', 10)->unique(); \$table->string('barang_nama', 100); \$table->integer('harga_beli'); \$table->integer('harga_jual'); \$table->timestamps(); // Mendefinisikan Foreign Key ke tabel m_kategori \$table->foreign('kategori_id')->references('kategori_id')->on('m_kategori'); }); } /** * Reverse the migrations. */ public function down(): void { Schema::dropIfExists('m_barang'); } };</pre>



b. t_penjualan

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('t_penjualan', function (Blueprint $table) {
            $table->id('penjualan_id');
            $table->unsignedBigInteger('user_id')->index();
            $table->string('pembeli', 50);
            $table->string('penjualan_kode', 20);
            $table->dateTime('penjualan_tanggal');
            $table->timestamps();

            $table->foreign('user_id')->references('user_id')->on('m_user');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('t_penjualan_detail');
    }
};
```

c. t_stok

```
return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('t_stok', function (Blueprint $table) {
            $table->id('stok_id'); // Primary Key
            $table->unsignedBigInteger('barang_id')->index(); // Foreign Key ke m_barang
            $table->unsignedBigInteger('user_id')->index(); // Foreign Key ke m_user
            $table->dateTime('stok_tanggal');
            $table->integer('stok_jumlah');
            $table->timestamps();

            // Definisi Foreign Key Constraints
            $table->foreign('barang_id')->references('barang_id')->on('m_barang');
            $table->foreign('user_id')->references('user_id')->on('m_user');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('t_stok');
    }
};
```



d. t_penjualan_detail

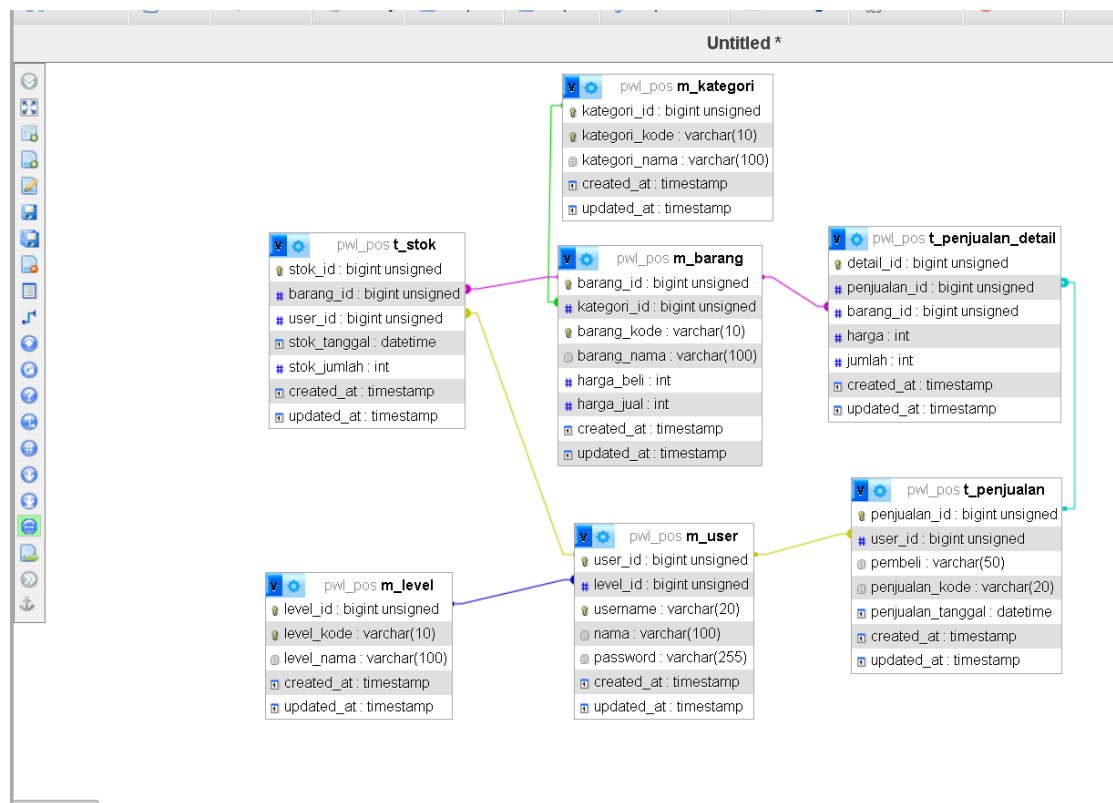
```
return new class extends Migration
{
    public function up(): void
    {
        Schema::create('t_penjualan_detail', function (Blueprint $table) {
            $table->id('detail_id'); // Primary Key
            $table->unsignedBigInteger('penjualan_id')->index(); // FK ke t_penjualan
            $table->unsignedBigInteger('barang_id')->index(); // FK ke m_barang
            $table->integer('harga');
            $table->integer('jumlah');
            $table->timestamps();

            // Definisi Foreign Key
            $table->foreign('penjualan_id')->references('penjualan_id')->on('t_penjualan');
            $table->foreign('barang_id')->references('barang_id')->on('m_barang');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('t_penjualan_detail');
    }
};
```

5

Jika semua file migrasi sudah di buat dan dijalankan maka bisa kita lihat tampilan designer pada phpMyAdmin seperti berikut





Praktikum Ke-3

Langkah	Jawaban/Deskripsi																								
1	Kita akan membuat file seeder untuk table m_level dengan mengetikkan perintah <pre>PS C:\laragon\www\JS3> php artisan make:seeder LevelSeeder</pre> INFO Seeder [C:\laragon\www\JS3\database\seeders\LevelSeeder.php] created successfully.																								
2	kita modifikasi file tersebut di dalam function run() <pre>namespace Database\Seeders; use Illuminate\Database\Console\Seeds\WithoutModelEvents; use Illuminate\Database\Seeder; use Illuminate\Support\Facades\DB; class LevelSeeder extends Seeder { /** * Run the database seeds. */ public function run(): void { \$data = [['level_id' =>1, 'level_kode' => 'ADM', 'level_nama' => 'Administrator'], ['level_id' =>2, 'level_kode' => 'MNG', 'level_nama' => 'Manager'], ['level_id' =>3, 'level_kode' => 'STF', 'level_nama' => 'Staff/Kasir'],]; DB::table('m_level')->insert(\$data); } }</pre>																								
3	Selanjutnya, kita jalankan file seeder untuk table m_level pada terminal <pre>PS C:\laragon\www\JS3> php artisan db:seed --class=LevelSeeder</pre> INFO Seeding database. <pre>PS C:\laragon\www\JS3></pre>																								
4	Ketika seeder berhasil dijalankan maka akan tampil data pada table m_level <table><tr><th></th><th>level_id</th><th>level_kode</th><th>level_nama</th><th>created_at</th><th>updated_at</th></tr><tr><td>☐ Edit ☐ Copy ☐ Delete</td><td>1</td><td>ADM</td><td>Administrator</td><td>NULL</td><td>NULL</td></tr><tr><td>☐ Edit ☐ Copy ☐ Delete</td><td>2</td><td>MNG</td><td>Manager</td><td>NULL</td><td>NULL</td></tr><tr><td>☐ Edit ☐ Copy ☐ Delete</td><td>3</td><td>STF</td><td>Staff/Kasir</td><td>NULL</td><td>NULL</td></tr></table>		level_id	level_kode	level_nama	created_at	updated_at	☐ Edit ☐ Copy ☐ Delete	1	ADM	Administrator	NULL	NULL	☐ Edit ☐ Copy ☐ Delete	2	MNG	Manager	NULL	NULL	☐ Edit ☐ Copy ☐ Delete	3	STF	Staff/Kasir	NULL	NULL
	level_id	level_kode	level_nama	created_at	updated_at																				
☐ Edit ☐ Copy ☐ Delete	1	ADM	Administrator	NULL	NULL																				
☐ Edit ☐ Copy ☐ Delete	2	MNG	Manager	NULL	NULL																				
☐ Edit ☐ Copy ☐ Delete	3	STF	Staff/Kasir	NULL	NULL																				
5	Sekarang kita buat file seeder untuk table m_user yang me-refer ke table m_level																								



	<pre>PS C:\laragon\www\JS3> php artisan make:seeder UserSeeder</pre>																								
6	Modifikasi file class UserSeeder seperti berikut <pre>class UserSeeder extends Seeder { /** * Run the database seeds. */ public function run(): void { \$data = [['user_id' => 1, 'level_id' => 1, 'username' => 'admin', 'nama' => 'Administrator', 'password' => Hash::make('12345'),], ['user_id' => 2, 'level_id' => 2, 'username' => 'manager', 'nama' => 'Manager', 'password' => Hash::make('12345'),], ['user_id' => 3, 'level_id' => 3, 'username' => 'staff', 'nama' => 'Staff/Kasir', 'password' => Hash::make('12345'),],]; DB::table('m_user')->insert(\$data); } }</pre>																								
7	Jalankan perintah untuk mengeksekusi class UserSeeder <pre>PS C:\laragon\www\JS3> php artisan db:seed --class=UserSeeder</pre> INFO Seeding database.																								
8	Perhatikan hasil seeder pada table m_user <table><tr><th></th><th>user_id</th><th>level_id</th><th>username</th><th>nama</th><th>password</th></tr><tr><td>☐ Edit ☐ Copy ☐ Delete</td><td>1</td><td>1</td><td>admin</td><td>Administrator</td><td>\$2y\$12\$5d7k52gKmXgp1s10</td></tr><tr><td>☐ Edit ☐ Copy ☐ Delete</td><td>2</td><td>2</td><td>manager</td><td>Manager</td><td>\$2y\$12\$3C4iIL2tjD/HSABfJsr</td></tr><tr><td>☐ Edit ☐ Copy ☐ Delete</td><td>3</td><td>3</td><td>staff</td><td>Staff/Kasir</td><td>\$2y\$12\$7YN/TI/dulx8LA4kdD</td></tr></table>		user_id	level_id	username	nama	password	☐ Edit ☐ Copy ☐ Delete	1	1	admin	Administrator	\$2y\$12\$5d7k52gKmXgp1s10	☐ Edit ☐ Copy ☐ Delete	2	2	manager	Manager	\$2y\$12\$3C4iIL2tjD/HSABfJsr	☐ Edit ☐ Copy ☐ Delete	3	3	staff	Staff/Kasir	\$2y\$12\$7YN/TI/dulx8LA4kdD
	user_id	level_id	username	nama	password																				
☐ Edit ☐ Copy ☐ Delete	1	1	admin	Administrator	\$2y\$12\$5d7k52gKmXgp1s10																				
☐ Edit ☐ Copy ☐ Delete	2	2	manager	Manager	\$2y\$12\$3C4iIL2tjD/HSABfJsr																				
☐ Edit ☐ Copy ☐ Delete	3	3	staff	Staff/Kasir	\$2y\$12\$7YN/TI/dulx8LA4kdD																				



9

masukkan data seeder untuk table yang lain, dengan ketentuan seperti berikut

No	Nama Tabel	Jumlah Data	Keterangan
1	m_kategori	5	5 kategori barang
2	m_barang	10	10 barang yang berbeda
3	t_stok	10	Stok untuk 10 barang
4	t_penjualan	10	10 transaksi penjualan
5	t_penjualan_detail	30	3 barang untuk setiap transaksi penjualan

a. m_kategori

```
PS C:\laragon\www\JS3> php artisan make:seeder KategoriSeeder
```

```
INFO Seeder [C:\laragon\www\JS3\database\seeders\KategoriSeeder.php] created successfully
```

```
class KategoriSeeder extends Seeder
{
    public function run(): void
    {
        $data = [
            ['kategori_kode' => 'ELK', 'kategori_nama' => 'Elektronik'],
            ['kategori_kode' => 'MKN', 'kategori_nama' => 'Makanan'],
            ['kategori_kode' => 'MNM', 'kategori_nama' => 'Minuman'],
            ['kategori_kode' => 'PCK', 'kategori_nama' => 'Pakaian'],
            ['kategori_kode' => 'KSM', 'kategori_nama' => 'Kosmetik'],
        ];
        DB::table('m_kategori')->insert($data);
    }
}
```

```
PS C:\laragon\www\JS3> php artisan db:seed --class=KategoriSeeder
```

```
INFO Seeding database.
```

				kategori_id	kategori_kode	kategori_nama	created_at	updated_at
☐	Edit	Copy	Delete	1	ELK	Elektronik	NULL	NULL
☐	Edit	Copy	Delete	2	MKN	Makanan	NULL	NULL
☐	Edit	Copy	Delete	3	MNM	Minuman	NULL	NULL
☐	Edit	Copy	Delete	4	PCK	Pakaian	NULL	NULL
☐	Edit	Copy	Delete	5	KSM	Kosmetik	NULL	NULL

b. m_barang

```
PS C:\laragon\www\JS3> php artisan make:seeder BarangSeeder
```

```
INFO Seeder [C:\laragon\www\JS3\database\seeders\BarangSeeder.php] created successfully
```



```
class BarangSeeder extends Seeder
{
    public function run(): void
    {
        $kategoriIds = DB::table('m_kategori')->pluck('kategori_id');
        $data = [];
        for ($i = 1; $i <= 10; $i++) {
            $data[] = [
                'kategori_id' => $kategoriIds[($i - 1) % 5],
                'barang_kode' => 'BRG' . str_pad($i, 3, '0', STR_PAD_LEFT),
                'barang_nama' => 'Barang ' . $i,
                'harga_beli' => 10000 * $i,
                'harga_jual' => 12000 * $i,
            ];
        }
        DB::table('m_barang')->insert($data);
    }
}
```

```
PS C:\laragon\www\JS3>
```

```
php artisan db:seed --class=BarangSeeder
```

```
INFO Seeding database.
```

			barang_id	kategori_id	barang_kode	barang_nama	harga_beli	harga_jual	created_at	updated_at
☐	Edit	Copy	Delete	1	1 BRG001	Barang 1	10000	12000	NULL	NULL
☐	Edit	Copy	Delete	2	5 BRG002	Barang 2	20000	24000	NULL	NULL
☐	Edit	Copy	Delete	3	2 BRG003	Barang 3	30000	36000	NULL	NULL
☐	Edit	Copy	Delete	4	3 BRG004	Barang 4	40000	48000	NULL	NULL
☐	Edit	Copy	Delete	5	4 BRG005	Barang 5	50000	60000	NULL	NULL
☐	Edit	Copy	Delete	6	1 BRG006	Barang 6	60000	72000	NULL	NULL
☐	Edit	Copy	Delete	7	5 BRG007	Barang 7	70000	84000	NULL	NULL
☐	Edit	Copy	Delete	8	2 BRG008	Barang 8	80000	96000	NULL	NULL
☐	Edit	Copy	Delete	9	3 BRG009	Barang 9	90000	108000	NULL	NULL
☐	Edit	Copy	Delete	10	4 BRG010	Barang 10	100000	120000	NULL	NULL

c. t_stok

```
PS C:\laragon\www\JS3> php artisan make:seeder StokSeeder
```

```
INFO Seeder [C:\laragon\www\JS3\database\seeders\StokSeeder.php] created successfully
```



```
class StokSeeder extends Seeder
{
    public function run(): void
    {
        $barangIds = DB::table('m_barang')->pluck('barang_id');
        $userId = DB::table('m_user')->first()->user_id ?? 1;

        foreach ($barangIds as $id) {
            DB::table('t_stok')->insert([
                'barang_id' => $id,
                'user_id' => $userId,
                'stok_tanggal' => now(),
                'stok_jumlah' => 100,
            ]);
        }
    }
}
```

```
PS C:\laragon\www\JS3> php artisan db:seed --class=StokSeeder
```

```
INFO Seeding database.
```

```
PS C:\laragon\www\JS3>
```

		stok_id	barang_id	user_id	stok_tanggal	stok_jumlah	created_at	updated_at
☐	Edit Copy Delete	1	1	1	2026-03-03 11:29:46	100	NULL	NULL
☐	Edit Copy Delete	2	6	1	2026-03-03 11:29:46	100	NULL	NULL
☐	Edit Copy Delete	3	3	1	2026-03-03 11:29:46	100	NULL	NULL
☐	Edit Copy Delete	4	8	1	2026-03-03 11:29:46	100	NULL	NULL
☐	Edit Copy Delete	5	4	1	2026-03-03 11:29:46	100	NULL	NULL
☐	Edit Copy Delete	6	9	1	2026-03-03 11:29:46	100	NULL	NULL
☐	Edit Copy Delete	7	5	1	2026-03-03 11:29:46	100	NULL	NULL
☐	Edit Copy Delete	8	10	1	2026-03-03 11:29:46	100	NULL	NULL
☐	Edit Copy Delete	9	2	1	2026-03-03 11:29:46	100	NULL	NULL
☐	Edit Copy Delete	10	7	1	2026-03-03 11:29:46	100	NULL	NULL

d. t_penjualan

```
PS C:\laragon\www\JS3> php artisan make:seeder PenjualanSeeder
```

```
INFO Seeder [C:\laragon\www\JS3\database\seeders\PenjualanSeeder.php] created successfully
```



```
class PenjualanSeeder extends Seeder
{
    public function run(): void
    {
        $userId = DB::table('m_user')->first()->user_id ?? 1;
        $barangIds = DB::table('m_barang')->pluck('barang_id');

        for ($i = 1; $i <= 10; $i++) {
            $penjualanId = DB::table('t_penjualan')->insertGetId([
                'user_id' => $userId,
                'pembeli' => 'Pelanggan ' . $i,
                'penjualan_kode' => 'TRX' . str_pad($i, 3, '0', STR_PAD_LEFT),
                'penjualan_tanggal' => now(),
            ]);

            // Input 3 barang per transaksi
            for ($j = 0; $j < 3; $j++) {
                $idx = ($i + $j) % 10;
                $barang = DB::table('m_barang')->where('barang_id', $barangIds[$idx])->first();

                DB::table('t_penjualan_detail')->insert([
                    'penjualan_id' => $penjualanId,
                    'barang_id' => $barang->barang_id,
                    'harga' => $barang->harga_jual,
                    'jumlah' => 1,
                ]);
            }
        }
    }
}
```

```
PS C:\laragon\www\JS3> php artisan db:seed --class=PenjualanSeeder
```

INFO Seeding database.

	penjualan_id	user_id	pembeli	penjualan_kode	penjualan_tanggal	created_at	updated_at
☐ Edit ☐ Copy ☐ Delete	1	1	Pelanggan 1	TRX001	2026-03-03 11:33:36	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	2	1	Pelanggan 2	TRX002	2026-03-03 11:33:37	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	3	1	Pelanggan 3	TRX003	2026-03-03 11:33:37	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	4	1	Pelanggan 4	TRX004	2026-03-03 11:33:37	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	5	1	Pelanggan 5	TRX005	2026-03-03 11:33:37	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	6	1	Pelanggan 6	TRX006	2026-03-03 11:33:37	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	7	1	Pelanggan 7	TRX007	2026-03-03 11:33:37	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	8	1	Pelanggan 8	TRX008	2026-03-03 11:33:37	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	9	1	Pelanggan 9	TRX009	2026-03-03 11:33:37	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	10	1	Pelanggan 10	TRX010	2026-03-03 11:33:37	NULL	NULL

e. t_detail_penjualan

```
PS C:\laragon\www\JS3> php artisan make:seeder PenjualanDetailSeeder
```

INFO Seeder [C:\laragon\www\JS3\database\seeders\PenjualanDetailSeeder.php] created successful



```
class PenjualanDetailSeeder extends Seeder
{
    public function run(): void
    {
        $userId = DB::table('m_user')->first()->user_id ?? 1;
        $barangIds = DB::table('m_barang')->pluck('barang_id');

        for ($i = 1; $i <= 10; $i++) {
            $penjualanId = DB::table('t_penjualan')->insertGetId([
                'user_id' => $userId,
                'pembeli' => 'Pelanggan ' . $i,
                'penjualan_kode' => 'TRX' . str_pad($i, 3, '0', STR_PAD_LEFT),
                'penjualan_tanggal' => now(),
                'created_at' => now(),
            ]);

            for ($j = 0; $j < 3; $j++) {
                $indexBarang = ($i + $j) % count($barangIds);
                $idBarang = $barangIds[$indexBarang];

                $hargaJual = DB::table('m_barang')->where('barang_id', $idBarang)->value('harga_jual');

                DB::table('t_penjualan_detail')->insert([
                    'penjualan_id' => $penjualanId,
                    'barang_id' => $idBarang,
                    'harga' => $hargaJual,
                    'jumlah' => rand(1, 5),
                    'created_at' => now(),
                ]);
            }
        }
    }
}
```

```
PS C:\laragon\www\JS3> php artisan db:seed --class=PenjualanDetailSeeder
```

INFO Seeding database.

		detail_id	penjualan_id	barang_id	harga	jumlah	created_at	updated_at
☐	Edit Copy Delete	1	1	6	72000	1	NULL	NULL
☐	Edit Copy Delete	2	1	3	36000	1	NULL	NULL
☐	Edit Copy Delete	3	1	8	96000	1	NULL	NULL
☐	Edit Copy Delete	4	2	3	36000	1	NULL	NULL
☐	Edit Copy Delete	5	2	8	96000	1	NULL	NULL
☐	Edit Copy Delete	6	2	4	48000	1	NULL	NULL

Praktikum Ke-4

Langkah	Jawaban/Deskripsi
1	Kita buat controller dahulu untuk mengelola data pada table m_level



	<pre>PS C:\laragon\www\JS3> php artisan make:controller LevelController</pre> INFO Controller [C:\laragon\www\JS3\app\Http\Controllers\LevelController.php] created successfully.																														
2	Kita modifikasi dulu untuk routing-nya, ada di PWL_POS/routes/web.php <pre><?php use App\Http\Controllers\LevelController; use Illuminate\Support\Facades\Route; Route::get('/', function () { return view('welcome'); }); Route::get('/level', [LevelController::class, 'index']);</pre>																														
3	Selanjutnya, kita modifikasi file LevelController untuk menambahkan 1 data ke table m_level <pre><?php namespace App\Http\Controllers; use Illuminate\Http\Request; use Illuminate\Support\Facades\DB; class LevelController extends Controller { public function index() { DB::insert('insert into m_level(level_kode,level_nama,created_at)values(?,?,?)', ['CUS','Pelanggan', now()]); return 'insert data baru berhasil'; } }</pre>																														
4	Kita coba jalankan di browser dengan url localhost/PWL_POS/public/level dan amati apa yang terjadi pada table m_level di database, screenshot perubahan yang ada pada table m_level <table><tr><th></th><th>level_id</th><th>level_kode</th><th>level_nama</th><th>created_at</th><th>updated_at</th></tr><tr><td>☐ Edit Copy Delete</td><td>1</td><td>ADM</td><td>Administrator</td><td>NULL</td><td>NULL</td></tr><tr><td>☐ Edit Copy Delete</td><td>2</td><td>MNG</td><td>Manager</td><td>NULL</td><td>NULL</td></tr><tr><td>☐ Edit Copy Delete</td><td>3</td><td>STF</td><td>Staff/Kasir</td><td>NULL</td><td>NULL</td></tr><tr><td>☐ Edit Copy Delete</td><td>4</td><td>CUS</td><td>Pelanggan</td><td>2026-03-03 12:58:29</td><td>NULL</td></tr></table>		level_id	level_kode	level_nama	created_at	updated_at	☐ Edit Copy Delete	1	ADM	Administrator	NULL	NULL	☐ Edit Copy Delete	2	MNG	Manager	NULL	NULL	☐ Edit Copy Delete	3	STF	Staff/Kasir	NULL	NULL	☐ Edit Copy Delete	4	CUS	Pelanggan	2026-03-03 12:58:29	NULL
	level_id	level_kode	level_nama	created_at	updated_at																										
☐ Edit Copy Delete	1	ADM	Administrator	NULL	NULL																										
☐ Edit Copy Delete	2	MNG	Manager	NULL	NULL																										
☐ Edit Copy Delete	3	STF	Staff/Kasir	NULL	NULL																										
☐ Edit Copy Delete	4	CUS	Pelanggan	2026-03-03 12:58:29	NULL																										
5	Selanjutnya, kita modifikasi lagi file LevelController untuk meng-update data di table m_level seperti berikut																														



```
class LevelController extends Controller
{
    public function index()
    {
        // DB::insert('insert into m_level(level_kode,level_nama,created_at)values(?,?,?)', ['CUS','Pelanggan', now()]);
        // return 'insert data baru berhasil';

        $row = DB::update('update m_level set level_nama = ? where level_kode = ?', ['Customer', 'CUS']);
        return 'Update data Berhasil. Jumlah data yang di update: ' . $row . ' baris';
    }
}
```

6

Kita coba jalankan di browser dengan url localhost/PWL_POS/public/level lagi dan amati apa yang terjadi pada table m_level di database, screenshot perubahan yang ada pada table m_level

localhost/JS3/public/level

Update data Berhasil. Jumlah data yang di update: 1 baris

level_id

level_kode

level_nama

created_at

updated_at

Edit

Copy

Delete

1

ADM

Administrator

NULL

NULL

Edit

Copy

Delete

2

MNG

Manager

NULL

NULL

Edit

Copy

Delete

3

STF

Staff/Kasir

NULL

NULL

Edit

Copy

Delete

4

CUS

Customer

2026-03-03 12:58:29

NULL

7

Kita coba modifikasi lagi file LevelController untuk melakukan proses hapus data

```
class LevelController extends Controller
{
    public function index()
    {
        // DB::insert('insert into m_level(level_kode,level_nama,created_at)values(?,?,?)', ['CUS','Pelanggan', now()]);
        // return 'insert data baru berhasil';

        // $row = DB::update('update m_level set level_nama = ? where level_kode = ?', ['customer', 'CUS']);
        // return 'Update data Berhasil. Jumlah data yang di update: ' . $row . ' baris';

        $row = DB::delete('delete from m_level where level_kode = ?', ['CUS']);
        return 'Delete data Berhasil. Jumlah data yang di hapus: ' . $row . ' baris';
    }
}
```

localhost/JS3/public/level

Delete data Berhasil. Jumlah data yang di hapus: 1 baris

level_id

level_kode

level_nama

created_at

updated_at

Edit

Copy

Delete

1

ADM

Administrator

NULL

NULL

Edit

Copy

Delete

2

MNG

Manager

NULL

NULL

Edit

Copy

Delete

3

STF

Staff/Kasir

NULL

NULL



- 8 Method terakhir yang kita coba adalah untuk menampilkan data yang ada di table m_level. Kita modifikasi file LevelController seperti berikut

```
class LevelController extends Controller
{
    public function index()
    {
        // DB::insert('insert into m_level(level_kode,
        // return 'insert data baru berhasil';

        // $row = DB::update('update m_level set level
        // return 'update data Berhasil. Jumlah data y

        // $row = DB::delete('delete from m_level wher
        // return 'Delete data Berhasil. Jumlah data y

        $data = DB::select('select * from m_level');
        return view('level', ['data' => $data]);
    }
}
```

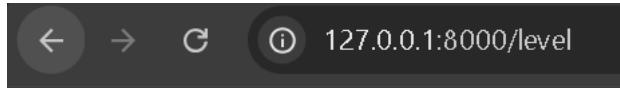
- 9 Coba kita perhatikan kode yang diberi tanda kotak merah, berhubung kode tersebut memanggil view('level'), maka kita buat file view pada VSCode di PWL_POS/resources/view/level.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Data Level Pengguna</title>
</head>
<body>
    <h1>Data Level Pengguna</h1>
    <table border="1" cellpadding="2" cellspacing="0">
        <tr>
            <th>ID</th>
            <th>Kode Level</th>
            <th>Nama Level</th>
        </tr>
        @foreach ($data as $d)
            <tr>
                <td>{{ $d->level_id }}</td>
                <td>{{ $d->level_kode }}</td>
                <td>{{ $d->level_nama }}</td>
            </tr>
        @endforeach
    </table>
</body>
</html>
```




10

Silahkan dicoba pada browser



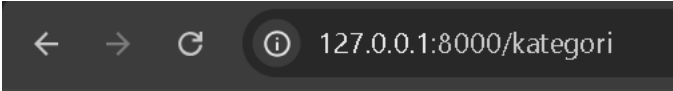
Data Level Pengguna

ID	Kode Level	Nama Level
1	ADM	Administrator
2	MNG	Manager
3	STF	Staff/Kasir

Praktikum Ke-5

Langkah	Jawaban/Deskripsi
1	Kita buat controller dahulu untuk mengelola data pada table m_kategori <pre>PS C:\laragon\www\JS3> php artisan make:controller KategoriController</pre> INFO Controller [C:\laragon\www\JS3\app\Http\Controllers\KategoriController.php] created successfully.
2	Kita modifikasi dulu untuk routing-nya, ada di PWL_POS/routes/web.php <pre>1 <?php 2 3 use App\Http\Controllers\KategoriController; 4 use App\Http\Controllers\LevelController; 5 use Illuminate\Support\Facades\Route; 6 7 8 Route::get('/', function () { 9 return view('welcome'); 10 }); 11 12 Route::get('/level', [LevelController::class, 'index']); 13 Route::get('/kategori', [KategoriController::class, 'index']);</pre>
3	Selanjutnya, kita modifikasi file KategoriController untuk menambahkan 1 data ke table m_kategori



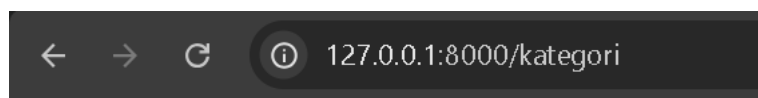
	<pre><?php namespace App\Http\Controllers; use Illuminate\Http\Request; use Illuminate\Support\Facades\DB; class KategoriController extends Controller { public function index() { \$data = ['kategori_kode' => 'SMK', 'kategori_nama' => 'Snack/Makana Ringan', 'created_at' => now()]; DB::table('m_kategori')->insert(\$data); return 'Insert data baru berhasil'; } }</pre>																																																														
4	Kita coba jalankan di browser  Insert data baru berhasil <table><tr><th></th><th></th><th></th><th>kategori_id</th><th>kategori_kode</th><th>kategori_nama</th><th>created_at</th><th>updated_at</th></tr><tr><td>☐</td><td></td><td></td><td></td><td>1</td><td>ELK</td><td>Elektronik</td><td>NULL</td><td>NULL</td></tr><tr><td>☐</td><td></td><td></td><td></td><td>2</td><td>MKN</td><td>Makanan</td><td>NULL</td><td>NULL</td></tr><tr><td>☐</td><td></td><td></td><td></td><td>3</td><td>MNM</td><td>Minuman</td><td>NULL</td><td>NULL</td></tr><tr><td>☐</td><td></td><td></td><td></td><td>4</td><td>PCK</td><td>Pakaian</td><td>NULL</td><td>NULL</td></tr><tr><td>☐</td><td></td><td></td><td></td><td>5</td><td>KSM</td><td>Kosmetik</td><td>NULL</td><td>NULL</td></tr><tr><td>☐</td><td></td><td></td><td></td><td>6</td><td>SMK</td><td>Snack/Makana Ringan</td><td>2026-03-03 13:36:59</td><td>NULL</td></tr></table>				kategori_id	kategori_kode	kategori_nama	created_at	updated_at	☐				1	ELK	Elektronik	NULL	NULL	☐				2	MKN	Makanan	NULL	NULL	☐				3	MNM	Minuman	NULL	NULL	☐				4	PCK	Pakaian	NULL	NULL	☐				5	KSM	Kosmetik	NULL	NULL	☐				6	SMK	Snack/Makana Ringan	2026-03-03 13:36:59	NULL
			kategori_id	kategori_kode	kategori_nama	created_at	updated_at																																																								
☐				1	ELK	Elektronik	NULL	NULL																																																							
☐				2	MKN	Makanan	NULL	NULL																																																							
☐				3	MNM	Minuman	NULL	NULL																																																							
☐				4	PCK	Pakaian	NULL	NULL																																																							
☐				5	KSM	Kosmetik	NULL	NULL																																																							
☐				6	SMK	Snack/Makana Ringan	2026-03-03 13:36:59	NULL																																																							
5	Selanjutnya, kita modifikasi lagi file KategoriController untuk meng-update data di table m_kategori seperti berikut																																																														



```
class KategoriController extends Controller
{
    public function index()
    {
        /*$data = [
            'kategori_kode' => 'SMK',
            'kategori_nama' => 'Snack/Makana Ringan',
            'created_at' => now()
        ];
        DB::table('m_kategori')->insert($data);
        return 'Insert data baru berhasil';*/

        $row = DB::table('m_kategori')->where('kategori_kode', 'SMK')->update(['kategori_nama' => 'Camilan']);
        return 'Update data berhasil. Jumlah data yang di update: '.$row.' baris';
    }
}
```

6 Kita coba jalankan di browser



Update data berhasil. Jumlah data yang di update: 1 baris

				kategori_id	kategori_kode	kategori_nama	created_at	updated_at
☐	Edit	Copy	Delete	1	ELK	Elektronik	NULL	NULL
☐	Edit	Copy	Delete	2	MKN	Makanan	NULL	NULL
☐	Edit	Copy	Delete	3	MNM	Minuman	NULL	NULL
☐	Edit	Copy	Delete	4	PCK	Pakaian	NULL	NULL
☐	Edit	Copy	Delete	5	KSM	Kosmetik	NULL	NULL
☐	Edit	Copy	Delete	6	SMK	Camilan	2026-03-03 13:36:59	NULL

7 Method terakhir yang kita coba adalah untuk menampilkan data yang ada di table m_kategori. Kita modifikasi file KategoriController seperti berikut

```
class KategoriController extends Controller
{
    public function index()
    {
        /*$data = [
            'kategori_kode' => 'SMK',
            'kategori_nama' => 'Snack/Makana Ringan',
            'created_at' => now()
        ];
        DB::table('m_kategori')->insert($data);
        return 'Insert data baru berhasil';*/

        // $row = DB::table('m_kategori')->where('kategori_kode', 'SMK')->update(['kategori_nama' => 'Camilan']);
        // return 'Update data berhasil. Jumlah data yang di update: '.$row.' baris';

        $data = DB::table('m_kategori')->get();
        return view('kategori', ['data' => $data]);
    }
}
```



- 8 Coba kita perhatikan kode yang diberi tanda kotak merah, berhubung kode tersebut memanggil view('kategori'), maka kita buat file view pada VSCode di PWL_POS/resources/view/kategori.blade.php

```
<!DOCTYPE html>
<html>
<head>
<title>Data Kategori Barang</title>
</head>
<body>
<h1>Data Kategori Barang</h1>
<table border="1" cellpadding="2" cellspacing="0">
<tr>
<th>ID</th>
<th>Kode Kategori</th>
<th>Nama Kategori</th>
</tr>
@foreach ($data as $d)
<tr>
<td>{{ $d->kategori_id }}</td>
<td>{{ $d->kategori_kode }}</td>
<td>{{ $d->kategori_nama }}</td>
</tr>
@endforeach
</table>
</body>
</html>
```

- 9 Silahkan dicoba pada browser dan amati apa yang terjadi.

← → ↻ ⓘ 127.0.0.1:8000/kategori

Data Kategori Barang

ID	Kode Kategori	Nama Kategori
1	ELK	Elektronik
2	MKN	Makanan
3	MNM	Minuman
4	PCK	Pakaian
5	KSM	Kosmetik
6	SMK	Camilan



Praktikum Ke-6

Langkah	Jawaban/Deskripsi
1	Kita buat file model untuk tabel m_user dengan mengetikkan perintah <pre>PS C:\laragon\www\JS3> php artisan make:model UserModel</pre> INFO Model [C:\laragon\www\JS3\app\Models\UserModel.php] created successfully.
2	Kita buka file UserModel.php dan modifikasi seperti berikut <pre><?php namespace App\Models; use Illuminate\Database\Eloquent\Factories\HasFactory; use Illuminate\Database\Eloquent\Model; class UserModel extends Model { use HasFactory; protected \$table = 'm_user'; protected \$primaryKey = 'user_id'; }</pre>
3	Kita buka file UserModel.php dan modifikasi seperti berikut <pre><?php namespace App\Models; use Illuminate\Database\Eloquent\Factories\HasFactory; use Illuminate\Database\Eloquent\Model; class UserModel extends Model { use HasFactory; protected \$table = 'm_user'; protected \$primaryKey = 'user_id'; }</pre>
4	Kita modifikasi route web.php untuk mencoba routing ke controller UserController



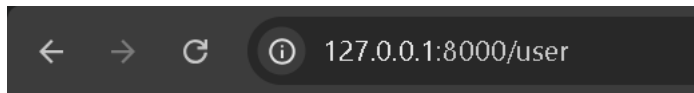
	<pre><?php use App\Http\Controllers\KategoriController; use App\Http\Controllers\LevelController; use App\Http\Controllers\UserController; use Illuminate\Support\Facades\Route; Route::get('/', function () { return view('welcome'); }); Route::get('/level', [LevelController::class, 'index']); Route::get('/kategori', [KategoriController::class, 'index']); Route::get('/user', [UserController::class, 'index']);</pre>
5	Sekarang, kita buat file controller UserController dan memodifikasinya seperti berikut <pre><?php namespace App\Http\Controllers; use App\Models\UserModel; use Illuminate\Support\Facades\DB; class UserController extends Controller { public function index() { \$user = UserModel::all(); return view('user', ['data' => \$user]); } }</pre>
6	Kemudian kita buat view user.blade.php



```
<!DOCTYPE html>
<html>
  <head>
    <title>Data User</title>
  </head>
  <body>
    <h1>Data User</h1>
    <table border="1" cellpadding="2" cellspacing="0">
      <tr>
        <th>ID</th>
        <th>Username</th>
        <th>Nama</th>
        <th>ID Level Pengguna</th>
      </tr>
      @foreach ($data as $d)
      <tr>
        <td>{{ $d->user_id }}</td>
        <td>{{ $d->username }}</td>
        <td>{{ $d->nama }}</td>
        <td>{{ $d->level_id }}</td>
      </tr>
      @endforeach
    </table>
  </body>
</html>
```

7

Jalankan di browser



Data User

ID	Username	Nama	ID Level Pengguna
1	admin	Administrator	1
2	manager	Manager	2
3	staff	Staff/Kasir	3

8

Setelah itu, kita modifikasi lagi file UserController



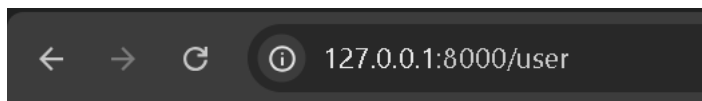
```
use App\Models\UserModel;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;

class UserController extends Controller
{
    public function index()
    {
        //tambah data user dengan Eloquent Model
        $data = [
            'username' => 'customer-1',
            'nama' => 'Pelanggan',
            'password' => Hash::make('12345'),
            'level_id' => 4
        ];
        UserModel::insert($data);

        $user = UserModel::all();
        return view('user', ['data' => $user]);
    }
}
```

9

Jalankan di browser



Data User

ID	Username	Nama	ID Level Pengguna
1	admin	Administrator	1
2	manager	Manager	2
3	staff	Staff/Kasir	3
9	customer-1	Pelanggan	5

10

Kita modifikasi lagi file UserController menjadi seperti berikut



```
<?php

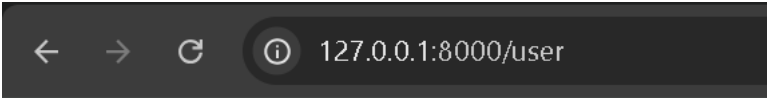
namespace App\Http\Controllers;

use App\Models\UserModel;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;

class UserController extends Controller
{
    public function index()
    {
        $data = [
            'nama' => 'Pelanggan Pertama',
        ];
        UserModel::where('username', 'customer-1')->update($data);

        $user = UserModel::all();
        return view('user', ['data' => $user]);
    }
}
```

11 Jalankan di browser



127.0.0.1:8000/user

Data User

ID	Username	Nama	ID Level Pengguna
1	admin	Administrator	1
2	manager	Manager	2
3	staff	Staff/Kasir	3
9	customer-1	Pelanggan Pertama	5

Soal Praktikum

No	Jawaban
1	Fungsi APP_KEY: APP_KEY digunakan sebagai kunci enkripsi untuk mengamankan data sensitif aplikasi, seperti data sesi (<i>session</i>), cookie, dan data lain yang perlu



	dienkripsi. Jika kunci ini tidak disetel, data yang dienkripsi tidak akan aman dan aplikasi mungkin mengalami kendala keamanan.
2	Cara men-generate nilai APP_KEY: Nilai tersebut dihasilkan dengan menjalankan perintah php artisan key:generate melalui terminal atau command prompt di direktori proyek Laravel.
3	File migrasi default Laravel: Secara default terdapat 4 file migrasi: • create_users_table: Untuk menyimpan data pengguna. • create_password_reset_tokens_table: Untuk menyimpan token pengaturan ulang kata sandi. • create_failed_jobs_table: Untuk mencatat antrian (<i>queue</i>) pekerjaan yang gagal. • create_personal_access_tokens_table: Untuk pengelolaan token akses API (Sanctum).
4	Tujuan \$table->timestamps(): Fungsi ini secara otomatis menciptakan dua kolom pada tabel, yaitu created_at (mencatat waktu data dibuat) dan updated_at (mencatat waktu data terakhir kali diperbarui).
5	Tipe data \$table->id(): Fungsi ini menghasilkan tipe data Big Integer yang bersifat <i>Unsigned</i> (tidak bernilai negatif) dan bertindak sebagai <i>Auto-Increment Primary Key</i> .
6	Perbedaan \$table->id() dan \$table->id('level_id'): \$table->id() akan menciptakan kolom primary key dengan nama default yaitu id. Sedangkan \$table->id('level_id') akan menciptakan kolom primary key dengan nama spesifik yaitu level_id.
7	Fungsi ->unique(): Digunakan untuk memastikan bahwa data di dalam kolom tersebut tidak boleh ada yang kembar (duplikat). Jika ada input data yang sama, database akan menolaknya.
8	Penggunaan unsignedBigInteger pada foreign key: Kolom level_id pada tabel m_user menggunakan unsignedBigInteger karena harus memiliki tipe data yang identik dengan primary key di tabel m_level (yang dihasilkan oleh \$table->id()). Hal ini merupakan syarat wajib agar relasi antar tabel (<i>foreign key</i>) dapat terbentuk dengan benar.
9	Tujuan Class Hash: Class Hash digunakan untuk melakukan enkripsi satu arah pada data sensitif seperti kata sandi. Kode Hash::make('1234') berfungsi untuk mengubah string



	"1234" menjadi kode acak (hash) yang aman sehingga kata sandi asli tidak tersimpan dalam bentuk teks biasa di database.
10	Kegunaan tanda tanya (?) pada Query Builder: Tanda tanya tersebut digunakan sebagai <i>placeholder</i> untuk <i>parameter binding</i> . Tujuannya adalah untuk meningkatkan keamanan aplikasi agar terhindar dari serangan <i>SQL Injection</i> .
11	Tujuan protected \$table dan \$primaryKey: - protected \$table = 'm_user';: Digunakan untuk mendefinisikan secara manual nama tabel yang digunakan oleh model, karena secara default Laravel akan mencari tabel dalam bentuk jamak (seperti users). - protected \$primaryKey = 'user_id';: Digunakan untuk mendefinisikan kolom mana yang menjadi primary key jika namanya bukan id.
12	Perbedaan Query Builder dan Eloquent ORM: <i>Query Builder</i> menyediakan cara yang lebih langsung dan cepat untuk berinteraksi dengan database menggunakan sintaksis PHP yang menyerupai SQL. Sementara itu, <i>Eloquent ORM</i> menggunakan pendekatan berbasis objek (<i>Object-Relational Mapper</i>) di mana setiap tabel direpresentasikan sebagai model, sehingga memudahkan pengelolaan relasi yang kompleks meskipun performanya sedikit lebih berat dibanding Query Builder.